

Introduction to Query Optimization

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. How Postgres Processes a Query
2. Advanced Joins
3. Hands-on Lab

Learning Objectives

By the end of this session, you will be able to:

- **The Query Lifecycle:** The journey from SQL string to result set.
- **Postgres Optimizer:** Understanding the Cost-Based Optimizer (CBO).
- **EXPLAIN ANALYZE:** "X-raying" your queries to find bottlenecks.
- **Advanced Join Algorithms:** Deep dive into Nested Loop, Hash Join, and Merge Join.

How Postgres Processes a Query

How Postgres Processes a Query

1. **Parser:** Checks syntax and validates tables/columns against the system catalog.
2. **Rewriter:** Handles Rules and Views (expands a View into its underlying query).
3. **Optimizer (Planner):** The "Brain." It generates multiple *Execution Plans* and selects the one with the lowest **Cost** based on table statistics ($\mathrm{pg_stats}$).
4. **Executor:** Performs the physical operations (Disk scans, Joins, Sorts) and returns the data.

Pro Tip for AI Students

The Optimizer uses probabilistic models to estimate row counts. If statistics are outdated, the "Optimal Plan" might actually be the slowest.

”X-Raying” Queries with EXPLAIN ANALYZE

EXPLAIN shows the plan. EXPLAIN ANALYZE **executes** the query and shows real-world timing.

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT * FROM customer WHERE last_name = 'Kelly';
```

Analyzing a Search Query

Key Metrics to Watch:

- **Seq Scan vs. Index Scan:** Are we brute-forcing the table or using the index?
- **Cost:** An abstract unit of work (CPU + I/O).
- **Actual Time:** Real execution time in milliseconds.
- **Shared Hit:** How many pages were found in the **Buffer Pool** (RAM) vs. Disk.

Mastering EXPLAIN Parameters

The command $\mathrm{EXPLAIN\ (ANALYZE,\ COSTS,\ VERBOSE,\ BUFFERS)}$ is the "Gold Standard" for database debugging. Here is what each keyword unlocks:

- $\mathrm{ANALYZE}$**:**
  - **Action:** Actually **executes** the query.
  - **Benefit:** Provides real-world timing and actual row counts. Without this, you only see the Optimizer's "guesses."
  - *Caution:* Be careful with $\mathrm{DELETE/UPDATE}$ queries (use a transaction!).
- COSTS**:**
  - **Action:** Shows the estimated startup and total cost (e.g., $0.00..100.50$).
  - **Benefit:** Helps you understand why the Optimizer chose one plan over another.

Mastering EXPLAIN Parameters

The command EXPLAIN (ANALYZE, COSTS, VERBOSE, BUFFERS) is the "Gold Standard" for database debugging. Here is what each keyword unlocks:

- VERBOSE:
  - **Action:** Displays additional details like the list of output columns (Output:) and schema-qualified table names.
  - **Benefit:** Essential for complex queries with many joins or subqueries.
- BUFFERS:
  - **Action:** Shows how many blocks were read from **RAM (Shared Hit)** vs. **Disk (Read)**.
  - **Benefit:** The most important metric for performance! High Disk Read = Slow query.

Case Study 1: The Sequential Scan

Let's look at a query that searches for a non-indexed column in the film table.

```
1 EXPLAIN (ANALYZE, COSTS, VERBOSE, BUFFERS)
2 SELECT * FROM film WHERE replacement_cost = 14.99;
```

The Header Line

Seq Scan on public.film (cost=0.00..100.50 rows=51 width=384)

- **Seq Scan:** The access method. Postgres reads the *entire* table because there is no Index on replacement_cost.
- **cost=0.00..100.50:**
  - 0.00: Startup cost (time to get the first row).
  - 100.50: Total estimated cost (units of disk I/O + CPU).
- **rows=51:** The Optimizer **estimates** that 51 films have this price.
- **width=384:** Average bytes per row. Important for memory (work_mem) calculation.

Execution Stats: Theory vs. Reality

The Execution Line

$$(\text{actual time}=0.009..0.567\ \text{rows}=51\ \text{loops}=1)$$

- **actual time=0.009..0.567:**
  - 0.009: Real time (ms) to find the first record.
  - 0.567: Real time (ms) to return all 51 records.
- **rows=51:** The **actual** number of rows returned. (In this case, the Optimizer's guess was 100% accurate!).
- **loops=1:** This scan was performed once (relevant for Joins/Nested Loops).

Execution Stats: Theory vs. Reality

Filter Cleanup

Filter: (film.replacement_cost = 14.99)
Rows Removed by Filter: 949

Insight: The DB had to check 1000 rows (949 + 51) just to find our data. This is inefficient!

The Buffer Strategy: I/O Efficiency

Why "Buffers" Matter?

Buffers: shared hit=88

- **shared hit=88:** 88 data blocks (Pages) were found in the **Shared Buffer** (RAM). 0 were read from Disk.
- **Impact:** This query is extremely fast because the data was already "warm" in memory.

The Buffer Strategy: I/O Efficiency

Planning vs. Execution Time

- **Planning Time (2.103 ms):** Time taken to find the ”best” plan.
- **Execution Time (0.591 ms):** Actual time spent fetching data.

Note: If Planning Time > Execution Time, the query is simple but the overhead is relatively high.

Reference Table for Indexing Strategy

Metric	What it tells you	Optimization Goal
Seq Scan	Reading full table	Add an Index to convert to Index Scan.
Shared Hit	Data in RAM	High is good. Avoid "Read" (Disk I/O).
Rows Removed	Data discarded	Lower is better. If high, Indexing is needed.
Cost	DB's effort estimate	Lowering cost usually lowers execution time.
Actual Rows	Real result count	Compare with 'rows' to check stats health.

Question for the class: Looking at the output provided: "Rows Removed by Filter: 949". If we add a B-Tree Index on replacement_cost, what will happen to this metric?

Case Study 2: Index Scan

Let's look at a query that searches for a indexed column in the film table.

```
1 CREATE INDEX idx_film_replacement_cost ON film(replacement_cost);
2
3 EXPLAIN (ANALYZE, COSTS, VERBOSE, BUFFERS)
4 SELECT * FROM film WHERE replacement_cost = 14.99;
```

After creating the Index, Postgres changed its strategy to a **two-step process**.

1. Bitmap Index Scan (The "Discovery" Phase)

-> Bitmap Index Scan on idx_film_replacement_cost (cost=0.00..4.66 rows=51)

- **Action:** Instead of visiting the table for every row found, Postgres scans the Index first.
- **Creation:** It creates a "Bitmap" (a map of bits) in memory where each bit represents a page that contains a matching row.
- **Efficiency:** It only hits **2 buffers** (shared hit=2) to find all 51 locations.

The Result of Indexing: Bitmap Heap Scan

2. Bitmap Heap Scan (The "Retrieval" Phase)

Bitmap Heap Scan on public.film (cost=4.67..84.40 rows=51)

- **Action:** Postgres visits the table (Heap) but **only** the pages marked in the Bitmap.
- **Recheck Cond:** Ensures the data still matches the condition (incase of concurrent updates).
- **Heap Blocks (exact=39):** It only needed to open 39 pages of data instead of the whole table.

- **Benefit:** This avoids "random I/O" by sorting the page requests, making it faster than a regular Index Scan for medium-sized results.

Before vs. After: The Verdict

Metric	Seq Scan (Before)	Bitmap Scan (After)	Improvement
Execution Time	0.591 ms	0.116 ms	≈ **5x Faster**
Total Cost	100.50	84.40	**Lower Cost**
Shared Hit	88 Buffers	41 Buffers	**53% Less I/O**
Rows Filtered	949 rows	0 rows	**Zero Waste**

- The dataset dvdrental is small. In a 100-million-row database, this change would be the difference between **15 minutes** and **1 second**.

Advanced Joins

Why Join Algorithms Matter?

In SQL, we just write JOIN. But under the hood, the Optimizer must choose a physical strategy to combine rows from two tables.

The three main strategies in PostgreSQL:

1. **Nested Loop Join:** The "Brute Force" approach.
2. **Hash Join:** The "Memory Search" approach.
3. **Merge Join:** The "Sorted Alignment" approach.

The Optimizer's Choice

Postgres chooses based on: Table size, availability of Indexes, and memory ($\mathrm{work_mem}$).

Nested Loop Join

Mechanism: For every row in Table A, scan Table B for a match.

```
1 EXPLAIN ANALYZE
2 SELECT f.title, i.inventory_id
3 FROM film f
4 JOIN inventory i ON f.film_id = i.film_id
5 WHERE f.film_id = 1;
```

Example: Joining a single film to its inventory

- **Best for:** Very small datasets or when the inner table has an **Index** on the join key.
- **Output in Explain:** Look for Nested Loop.
- **Efficiency:** Very high if the inner side is an *Index Scan*.
- **Complexity:** $O(N \times M)$.
- **Analogy:** Like looking through a physical folder (Outer) and for every paper, searching for a match in a filing cabinet (Inner).

The Core Mechanics of Nested Loop Join

At its heart, a Nested Loop is exactly like a **nested for-loop** in programming.

The Algorithm

```
for row_outer in table_outer: # Outer Loop
    # Potentially use an Index here!
    for row_inner in table_inner: # Inner Loop
        if row_outer.key == row_inner.key:
            emit_joined_row(row_outer, row_inner)
```

Key Definitions:

- **Outer Table (Driving Table):** Scanned once.
- **Inner Table (Driven Table):** Scanned N times (where N is the number of rows filtered from the Outer Table).

Who is Outer? Who is Inner?

The Optimizer doesn't flip a coin. It makes a **Cost-Based Decision** based on two golden rules:

1. **Rule 1: Smallest "Effective" Table becomes OUTER**
   - It looks at **Cardinality** (Estimated rows after WHERE filters).
   - *Logic:* If you only have to loop 5 times (Outer), it's better than looping 1000 times.
2. **Rule 2: The Best "Lookup" Table becomes INNER**
   - The Optimizer checks for **Index Availability**.
   - *Logic:* If the Inner table has an Index on the Join Key, each "inner loop" becomes a lightning-fast **Index Seek** instead of a full scan.

The Planner's Math (Simplified)

The Planner estimates the "Total Cost" using this fundamental formula:

$$Cost_{total} = Cost_{outer_scan} + (Rows_{outer} \times Cost_{inner_scan})$$

Why this explains the behavior:

- If $Rows_{outer}$ is small (e.g., 5), the total cost remains low.
- If $Cost_{inner_scan}$ is an **Index Scan** (e.g., 0.25), the total cost remains low.
- **The Disaster Scenario:** If $Rows_{outer} = 10^6$ and $Cost_{inner_scan} = 100$ (Seq Scan), the cost explodes!

*Conclusion: To optimize a Nested Loop, always ensure your Inner table's Join Key is **Indexed**.*

Lab Exercise 1: Triggering Nested Loop Join

The Scenario

Find the name and rental history for a specific customer ($\mathrm{ID} = 1$).

```sql
-- Step 1: Execute this and observe the "Nested Loop" node
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.first_name, c.last_name, r.rental_date
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
WHERE c.customer_id = 1;
```

Analysis Questions:

- **Identify the Roles:** Look at the plan tree. Which table is the **Outer** (top) and which is the **Inner** (bottom)?
- **Why Nested Loop?** How many rows did the $\mathrm{customer}$ table return? Why does this make Hash Join "too expensive"?
- **Efficiency:** Look at loops in the Inner Table node. Why is it 1?

Hash Join

Mechanism: 1. Build a **Hash Table** in RAM from the smaller table. 2. Scan the larger table and ”probe” the hash table.

```
1 EXPLAIN ANALYZE
2 SELECT c.first_name, r.rental_date
3 FROM customer c
4 JOIN rental r ON c.customer_id = r.customer_id;
```

Example: Joining large tables without specific filters

- **Best for:** Large tables, equality joins ($=$), and no usable indexes.
- **Output in Explain:** $\mathrm{Hash\ Join} + \mathrm{Hash}$.
- **Limitation:** Requires $\mathrm{work_mem}$. If the hash table is too big, it spills to disk (slow).
- **Complexity:** $O(N + M)$ – very efficient.

The Anatomy of Hash Join (The 2-Phase Strategy)

Unlike Nested Loop, Hash Join does not scan the inner table repeatedly. It uses a **Divide and Conquer** strategy through Hashing.

Phase 1: BUILD (Preparing the Hash Map)

Scan the **Inner Table** once. Apply a Hash Function to the Join Key and store the row in a Hash Table (in-memory).

Phase 2: PROBE (The Look-up)

Scan the **Outer Table** once. For each row, hash its Join Key and check the Hash Table for a match.

Ideal Complexity: $O(N + M)$ – This is linear and much faster than $O(N \times M)$ for large tables.

Hash Join Pseudocode

```
# -- BUILD phase
hash_table = {}
for row_inner in inner_table:
    key = hash_func(row_inner.join_key)
    hash_table[key] = row_inner

# -- PROBE phase
for row_outer in outer_table:
    key = hash_func(row_outer.join_key)
    if key in hash_table:
        emit_joined_row(row_outer, hash_table[key])
```

How Hash Join Executes

Picking Inner vs. Outer (The Golden Rule)

In Hash Join, the roles of Inner and Outer are strictly defined by memory:

- **INNER = The Smaller Table:** Must ideally fit entirely into $\mathrm{work_mem}$ (RAM).
- **OUTER = The Larger Table:** Only scanned once to “probe” the memory.

How to read the Execution Plan:

```
Hash Join
  -> Seq Scan on big_table (OUTER - Probe)
  -> Hash
       -> Seq Scan on small_table (INNER - Build)
```

Constraints & The ”work_mem” Lifeblood

Hash Join is powerful but has strict requirements:

1. **Equality Only ($=$):** It relies on Hashing. It **cannot** handle range joins like $<$, $>$, BETWEEN or LIKE.
2. **Memory Dependence (work_mem):** This is the memory allocated per operation.

Good: In-Memory	Bad: Spill to Disk
Batches: 1 Memory Usage: 8MB → **Ultra fast.**	Batches: 8 Disk Usage: 256MB → **Slow I/O bottleneck.**

If Batches > 1, Postgres has to split the hash table into files on disk, killing performance.

When does the Optimizer choose Hash Join?

The Optimizer runs a cost-benefit analysis. It will choose Hash Join when:

- **No Suitable Indexes:** If Nested Loop is too expensive because it would require multiple full scans.
- **Unordered Data:** If the data isn't sorted (making Merge Join expensive due to SORT costs).
- **Equality Condition:** The join is based on $\mathrm{a.id} = \mathrm{b.id}$.

Cost Model Estimate

$$Cost \approx Cost(Scan_Both) + Cost(Build_Hash) + Cost(Cpu_Probe)$$

Lab Exercise 2: Triggering Hash Join

Goal: Observe how Postgres handles joining **whole tables** without restrictive filters.

The Scenario

Create a full report linking every Film to its Language. (Both tables are scanned fully).

```
-- Run this and look for "Hash Join" and "Hash" nodes
EXPLAIN (ANALYZE, BUFFERS)
SELECT f.title, l.name as language
FROM film f
JOIN language l ON f.language_id = l.language_id;
```

Analysis Questions:

- **Identify Phases:** Which table is being Hashed (Built)? Which table is being Probed?
- **Memory Check:** Look at the Hash node. What is the Memory Usage?
- **Hypothesis:** What happens if you run SET work_mem = '64kB'; and run it again? (Try it!)

Merge Join

Mechanism: Sort both tables by the join key, then walk through them together.

```sql
-- We can force a Merge Join by disabling others for demonstration
SET enable_hashjoin = off;
SET enable_nestedloop = off;

EXPLAIN ANALYZE
SELECT f.title, i.inventory_id
FROM film f
JOIN inventory i ON f.film_id = i.film_id;

RESET enable_hashjoin; RESET enable_nestedloop;
```

Example: Joining sorted data

- **Best for:** Very large joins where data is already sorted by an Index.
- **Output in Explain:** Merge Join + Sort.
- **Advantage:** Very stable for "Big Data" as it doesn't need to fit everything in RAM like Hash Join.

The Anatomy of Merge Join (The Zipper Approach)

Merge Join works exactly like zipping two interlocking rows of teeth on a zipper.

The Core Prerequisite: SORTED INPUTS

Both inputs **must** be sorted by the Join Key before the merge phase begins.

Mechanism:

- Pointers are placed at the start of both tables.
- Compare keys. If equal, join. If not, advance the pointer with the smaller value.
- **No Random Access:** It reads data sequentially.

Ideal Complexity: $O(N + M)$ (excluding the cost of sorting).

Merge Join Pseudocode

```
# Pre-condition: Table A and B are sorted by Join Key
i = 0, j = 0

while i < len(A) and j < len(B):
    if A[i].key == B[j].key:
        emit_joined_row(A[i], B[j])
        # Handle duplicates (mark position, scan B, etc.)
    elif A[i].key < B[j].key:
        i += 1  # Advance A to catch up
    else:
        j += 1  # Advance B to catch up
```

The "Zipper" Logic

Note: It stops immediately once one list is exhausted.

The Hidden Cost: Where does the Sort come from?

The performance of Merge Join depends entirely on **how** the data gets sorted.

Scenario A: The "Free" Lunch

Source: B-Tree Index.

- Data is logically already sorted via Index.
- **Cost:** Very Low.
- **Plan:** $\text{Index Scan} \rightarrow \text{Merge Join}$.

Scenario B: The Expensive Bill

Source: Explicit Sort.

- Postgres must read the whole table and sort it in memory/disk.
- **Cost:** High ($O(N \log N)$).
- **Plan:** $\text{Seq Scan} \rightarrow \text{Sort} \rightarrow \text{Merge Join}$.

When does the Optimizer choose Merge Join?

Merge Join is the preferred strategy in specific "Big Data" scenarios:

1. **Data exceeds RAM:** Unlike Hash Join, Merge Join doesn't need to hold a Hash Table in memory. It streams data.
2. **Useful Indexes Exist:** If you join on Primary Keys (which are indexed), the sort step is skipped.
3. **Sorted Output Required:** If your query has $\mathrm{ORDER\ BY}$, Merge Join provides the order "for free".
4. **Non-Equality Joins (Partial):** It can handle some range conditions better than Hash Join.

AI Insight

Great for **Time-Series Data** (e.g., Sensor Logs) where data is naturally inserted and queried in chronological order.

Lab Exercise 3: Triggering Merge Join

Goal: Force a Merge Join by utilizing sorted Indexes or explicit ordering.

The Scenario

Join Films and Inventory. Since both are linked by $\mathrm{film_id}$ (which is indexed and sorted), this is a candidate for Merge Join.

```sql
-- Hint: Sometimes we must nudge the optimizer if tables are small
SET enable_hashjoin = OFF;
SET enable_nestedloop = OFF;

EXPLAIN (ANALYZE, BUFFERS)
SELECT f.film_id, f.title, i.inventory_id
FROM film f
JOIN inventory i ON f.film_id = i.film_id
ORDER BY f.film_id; -- Explicit sort requirement
```

Analysis Questions:

- **Structure:** Do you see a Sort node? Or did it use $\mathrm{Index\ Scan}$ to get sorted data for free?
- **Compare:** Reset the settings ($\mathrm{RESET\ ALL;}$) and remove the $\mathrm{ORDER\ BY}$. Does it switch back to Hash Join? Why?

Summary: Comparison of Join Strategies

Criteria	Nested Loop Join	Hash Join	Merge Join
Analogy	"Smart Brute Force"	"In-Memory Hash Map"	"The Zipper"
Mechanism	Loop inside a loop	Build (Hash Table) $\rightarrow$ Probe	Sort $\rightarrow$ Merge
Complexity	$O(N \times \log M)$ (w/ Index)	$O(N + M)$	$O(N + M)$ (if sorted)
Memory	Very Low (Streaming)	High ($\mathrm{work_mem}$)	Low (Streaming)
Operators	Any ($=, <, >$, BETWEEN)	Equality only ($=$)	Equality only ($=$)
Use Case	**OLTP:** Web apps, specific lookups	**OLAP:** Reports, Data Warehouse	**Big Data:** Massive streams, Time-series

Rules of Thumb

- Index + Small Data $\rightarrow$ **Nested Loop**.
- No Index + Large Data $\rightarrow$ **Hash Join**.
- Already Sorted + Huge Data $\rightarrow$ **Merge Join**.

Tip for AI Engineers

When preparing large Training Sets from CSVs (ETL), Postgres will likely choose **Hash Join**. Ensure your $\mathrm{work_mem}$ is large enough to avoid spilling to disk (which kills training pipeline speed).

Brain Teaser: Will this be a Nested Loop Join?

Look at the following query. Based on what we learned, analyze if Postgres will use **Nested Loop** or **Hash Join**.

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT c.first_name, c.last_name, r.rental_date
3 FROM rental r
4 JOIN customer c ON r.customer_id = c.customer_id
5 WHERE r.inventory_id = 1525;
```

Critical Thinking Questions

1. $\mathrm{r.inventory_id} = 1525$ returns **5 rows**. Why isn't this "small enough" for a Nested Loop?
2. The $\mathrm{customer}$ table has **599 rows**. How does this influence the decision?
3. **The "What If":** If the $\mathrm{customer}$ table had **1 million rows**, would Postgres change its mind? Why?

Hands-on Lab

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

```sql
-- Force a Sequential Scan by disabling Index Scans
SET enable_indexscan = off;
EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';

-- Re-enable and compare the Cost and Time
SET enable_indexscan = on;
EXPLAIN ANALYZE SELECT * FROM rental WHERE rental_date > '2005-05-24';
```

Task 2: Join Strategy Analysis

- Run a 3-way join: $\text{customer} \bowtie \text{rental} \bowtie \text{inventory}$.
- Check which join algorithm Postgres chose.
- **Challenge:** Run ANALYZE on the tables and see if the Plan changes.

Summary

- **The Optimizer is a Statistician:** It relies on $\mathrm{pg_stats}$. If your data changes drastically, run $\mathrm{ANALYZE}$.
- **Execution Plans matter:** A bad plan can make a query 1000x slower.
- **Nested Loop** (Index-heavy), **Hash Join** (Memory-heavy), and **Merge Join** (Sort-heavy) are the three pillars of data retrieval.

Preview for Session 4: We will move from basic queries to **Window Functions** for advanced data analytics.